

Python - Chapitre 3

Algorithmes élémentaires et complexité

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 1

Combien de tests $x > 0$ ce programme effectue-t-il ?

```
positifs = 0
for x in [3, -1, 0, 4, 2]:
    if x > 0:
        positifs += 1
print(positifs)
```

- A. 3
- B. 5
- C. 2
- D. Cela dépend du nombre de valeurs positives

Bonne réponse : B. 5

Le programme affiche 3, mais il teste chacune des cinq valeurs. Le nombre de résultats positifs et le nombre de tests sont différents.

Question 2

Quelle borne décrit le plus précisément le nombre d'incrémentations quand n grandit ?

```
n = 100
total = 0
for i in range(n):
    total += 1
for j in range(n):
    total += 1
```

- A. $O(1)$
- B. $O(\log n)$
- C. $O(n^2)$
- D. $O(n)$

Bonne réponse : D. $O(n)$

Les deux boucles se suivent : elles font $n + n = 2n$ incrémentations. Le travail est linéaire, donc en $O(n)$.

Python - Chapitre 3

Algorithmes élémentaires et complexité

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 3

Quelle condition permet à la dichotomie d'écarter une moitié de la liste ?

- A. La liste est triée selon l'ordre utilisé pour comparer
- B. La liste ne contient aucun doublon
- C. La taille de la liste est paire
- D. La cible est forcément présente

Bonne réponse : A. La liste est triée selon l'ordre utilisé pour comparer

Le tri garantit que les valeurs d'un côté du milieu sont toutes trop petites ou toutes trop grandes. Une cible absente et des doublons restent possibles.

Question 4

Quelle valeur prend gauche après cette première étape ?

```
valeurs = [2, 5, 8, 12, 16, 23, 38]
cible = 23
gauche, droite = 0, 6
milieu = (gauche + droite) // 2
if valeurs[milieu] < cible:
    gauche = milieu + 1
print(gauche)
```

- A. 3
- B. 5
- C. 4
- D. 6

Bonne réponse : C. 4

milieu vaut 3 et valeurs[3] vaut 12. Comme 12 < 23, on écarte les positions 0 à 3. La nouvelle borne gauche vaut 4.

Python - Chapitre 3

Algorithmes élémentaires et complexité

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 5

Combien de paires ce programme examine-t-il ?

```
n = 4
paires = 0
for i in range(n):
    for j in range(i + 1, n):
        paires += 1
print(paires)
```

- A. 4
- B. 6
- C. 12
- D. 16

Bonne réponse : B. 6

Les paires sont (0,1), (0,2), (0,3), (1,2), (1,3) et (2,3). Chaque paire de positions distinctes est comptée une fois : $n(n - 1)/2$.

Question 6

Sans doublon et avec le coût moyen du hachage, quelle borne est la plus précise pour n entiers ?

```
valeurs = [4, 1, 7, 9]
vus = set()
for valeur in valeurs:
    if valeur in vus:
        break
    vus.add(valeur)
```

- A. $O(n)$ en moyenne
- B. $O(1)$ pour tout le parcours
- C. $O(n \log n)$ en moyenne
- D. $O(n^2)$ en moyenne

Bonne réponse : A. $O(n)$ en moyenne

Sans doublon, la boucle fait n tours. Chaque tour réalise un test et un ajout de coût moyen $O(1)$, soit $O(n)$ au total. L'ensemble occupe $O(n)$ mémoire.

Python - Chapitre 3

Algorithmes élémentaires et complexité

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 7

Combien de fois ce code construit-il l'ensemble du catalogue ?

```
catalogue = [10, 20, 30]
requetes = [30, 99, 10]
reponses = []
for cible in requetes:
    reponses.append(cible in set(catalogue))
```

- A. 0
- B. 1
- C. 3
- D. 9

Bonne réponse : C. 3

set(catalogue) se trouve dans la boucle : il est exécuté une fois par requête, donc trois fois. Construire l'ensemble avant la boucle évite ce travail répété.

Question 8

Pour trouver deux articles distincts, que doit-on renvoyer pour prix = [5] et budget = 10 ?

- A. (0, 0)
- B. None
- C. (0, 1)
- D. 5

Bonne réponse : B. None

Un seul article ne permet pas de former deux indices distincts. Dans la solution par dictionnaire, chercher le complément avant l'ajout empêche de réutiliser le même indice.

Python - Chapitre 3

Algorithmes élémentaires et complexité

Quiz de révision | 10 questions | Une seule bonne réponse par question

Question 9

La fenêtre [2, 4, 6] devient [4, 6, 8]. Quelle est sa nouvelle somme ?

```
total = 2 + 4 + 6
total = total - 2 + 8
print(total)
```

- A. 12
- B. 20
- C. 24
- D. 18

Bonne réponse : D. 18

La somme passe de 12 à $12 - 2 + 8 = 18$. On retire la valeur sortante et on ajoute l'entrante, sans recalculer toute la fenêtre. La moyenne vaut 6.

Question 10

Pour n valeurs et une largeur k valide, combien de moyennes la fenêtre glissante renvoie-t-elle ?

- A. $n - k + 1$
- B. n
- C. k
- D. $n - k$

Bonne réponse : A. $n - k + 1$

Le début d'une fenêtre va de l'indice 0 à l'indice $n - k$ inclus. Cela donne $n - k + 1$ moyennes. La sortie occupe cet espace, même avec $O(1)$ auxiliaire.